

Wine Variety Prediction

<https://www.kaggle.com/datasets/zynicide/wine-reviews>

Introduction

In this report I will be examining a dataset consisting of wine reviews. Using several machine learning methods, the variety of the wine will be predicted based on the reviews or other data points provided in the dataset. I have broken the methods used down into three distinct categories.

- Word Count predictions – The features are the word token counts taken from the review descriptions.
- Sequence predictions – The features are the sequence of word tokens taken from the review descriptions.
- Point predictions – The features are all the extra numerical or categorical datapoints included in the dataset.

Feature Engineering

The dataset consists of 13 columns.

- country
- description
- designation
- points
- price
- province
- region_1
- region_2
- taster_name
- taster_twitter_handle
- title
- variety
- winery

The first action I performed on the dataset was to find the top 10 varieties of wine and remove the rest from the dataset. This brought the dataset from 129,971 rows to 71,322 rows. My intention was to simplify the problem by decreasing the number of labels being predicted. This also guarantees that none of the labels have an insufficient amount of data for the models to train on. From here, we can eliminate a few of the columns as they have too many null values or their information is irrelevant.

- designation – 22,214 null values
- region_1 – 10,092 null values
- region_2 – 34,659 null values

These combine for a total of 46,380 rows with null values in at least one of these columns which is nearly all the rows we have left.

- `taster_name` – We are not interested in making predictions based on who the reviewer is.
- `taster_twitter_handle` – Same as `taster_name`.
- `title` – This column has 65,693 unique values which makes it too specific to be useful.
- `winery` – I would include this one, but it crashes VSCode when I put it into `pd.get_dummies` because it has 11,782 unique values. I could encode it another way, but I want to remain consistent in how I am handling the data, so I am just leaving it out.

Now we just have the variety and five feature columns. We use just the description column for Word Count and Sequence predictions. This leaves Point predictions with country, points, price, and province. For every method, sklearn's LabelEncoder is used to encode variety.

Further feature engineering is performed on each category of machine learning methods individually.

- Sequence predictions
 - Recurrent Neural Network – Descriptions are tokenized using keras's Tokenizer class. A token is assigned to every word included in any description and then our feature vectors are generated by converting each sentence to a sequence of tokens. Each sequence is then padded to match the length of the longest sequence.
 - Transformer – I had trouble figuring out how to build my own transformer, so I used the `simpletransformers` library to do the heavy lifting for me. All I had to do was rename the description and variety columns to 'text' and 'labels' and feed them to the premade `ClassificationModel` which did the rest of the work for me.
- Word Count predictions
 - sklearn's `CountVectorizer` was used to generate sparse word count matrices for each row in the description column. `CountVectorizer` provides a few different hyperparameters when generating its counts. I set it to remove stop words as this provided a noticeable improvement in results. The n-gram range was tuned for each model. The maximum document frequency was set to 0.75.
- Point predictions
 - I assume that points are on a scale from 0-100 and normalize them accordingly (divided by 100). Price is normalized by dividing by the maximum price in the dataset. Country and province are converted to one-hot encoding using pandas' `get_dummies` function.

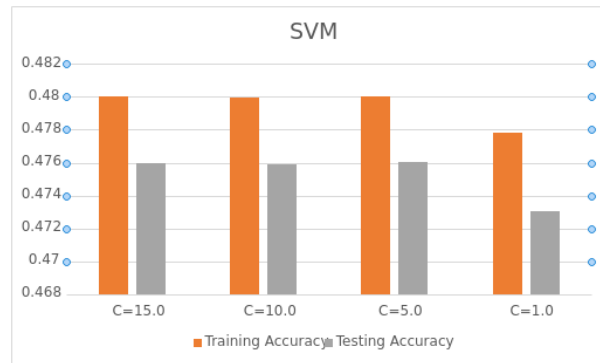
Every dataset is then split into a test and training set and fed to the models.

Models and Testing

For testing I will only be using training and test set accuracy to keep things simple. I recorded precision, recall, and F1 for the point prediction but I already have a significant amount of analysis using just the accuracy metric.

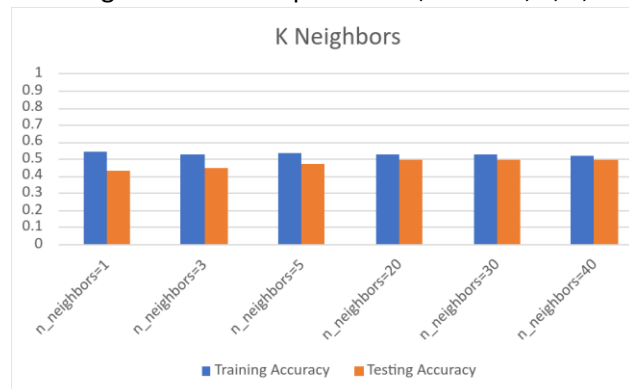
Point predictions

- Support Vector Machine – sklearn's LinearSVC is used. I set the number of max iterations to an arbitrarily large number (10,000) so that it always converges. This leaves C (the regularization parameter) as the only hyperparameter that I will be tuning. I tried 1, 5, 10, and 15 for C. They converge in 72, 104, 117, and 110 iterations, respectively. Here are my testing results for each C.



The training accuracy ranges between 0.478-0.48. The testing accuracy is between 0.473-0.476. Given this, I presume the primary factor in maximizing accuracy is reaching convergence. I also conclude that this accuracy is about as close to optimal as any model can get given that the other point predictions are only ever marginally better.

- K-Nearest Neighbors – sklearn's KNeighborsClassifier is used. The only hyperparameter considered is the number of neighbors. For this parameter, I tried 1, 3, 5, and 20.



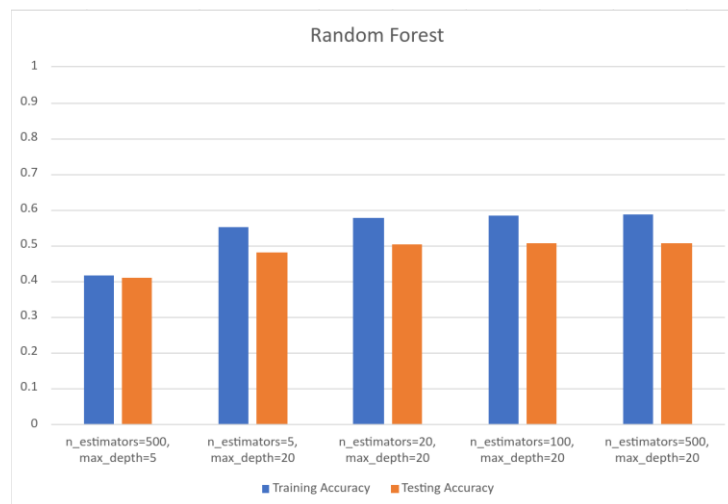
Increasing the number of neighbors notably increases the testing accuracy up until $n_neighbors=20$, at which point the improvement continues but at a much slower rate. Something notable about these results is that the training accuracy for $n_neighbors=1$ should be 1.00 but is instead 0.54219421. This means there are many points with identical locations but different varieties. This explains why the point prediction models struggle to achieve accuracy greater than 0.5.

- Decision Tree – sklearn's DecisionTreeClassifier is used. The only hyperparameter considered is the maximum depth of the tree. For this parameter, I tried 5, 20, 100, and 1000.



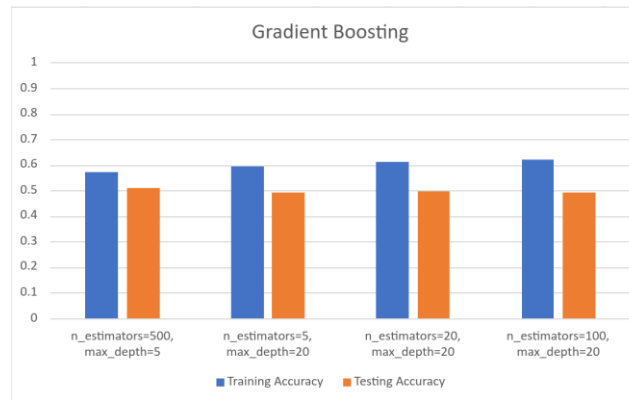
The testing accuracy increases significantly between 5 and 20, marginally between 20 and 100, and then completely levels off between 100 and 1000 as the dataset has been completely separated. Again, the training accuracy should be 1.00 for max_depth=1000 but is only 0.619.

- Random Forest – sklearn's RandomForestClassifier is used. I consider two hyperparameters, number of estimators and max depth of each estimator. I tried each of the parameter sets in the chart below.



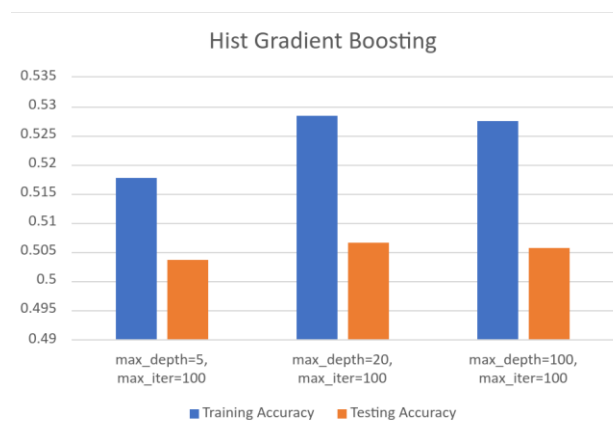
Based on these results, increasing the number of estimators is a reliable way to increase the testing accuracy up to 100 estimators, at which point the improvements tend to plateau towards 500 estimators. However, the test with 500 estimators of max depth 5 performs the worst of the group which implies that it is optimal to strike a balance between number of estimators and max depth rather than simply increasing the number of estimators repeatedly.

- Gradient Boosting – sklearn's GradientBoostingClassifier is used. I consider two hyperparameters, number of estimators and max depth of each estimator. I tried each of the parameter sets in the chart below.



The testing accuracy hovers around 0.5 for all results. This is likely because gradient boosting fits the data very quickly and not much improvement is made by increasing the initial parameters past `n_estimators=5` and `max_depth=20`.

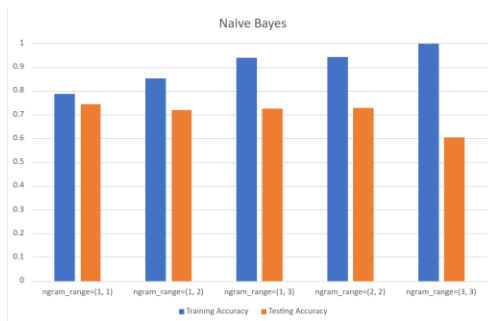
- Hist Gradient Boosting – sklearn's `HistGradientBoostingClassifier` is used. According to sklearn, `HistGradientBoostingClassifier` is a “Histogram-based Gradient Boosting Classification Tree” and is “much faster than `GradientBoostingClassifier` for big datasets (`n_samples >= 10 000`).” I set the max iterations to 100 and consider one hyperparameter, max depth. For this parameter I tried 5, 20, and 100.



The testing accuracy hovers around 0.5 for all results. This is likely because gradient boosting fits the data very quickly and not much improvement is made by increasing the max depth past 5.

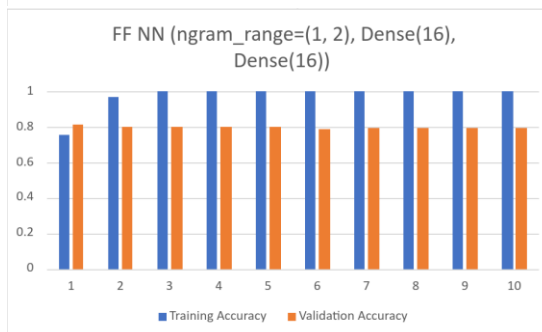
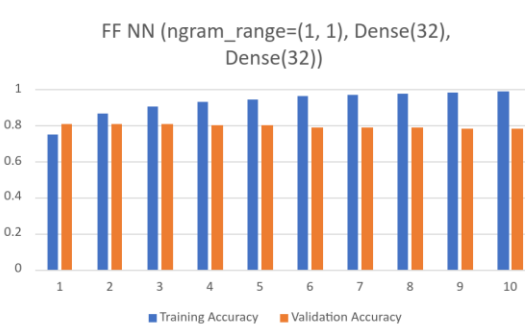
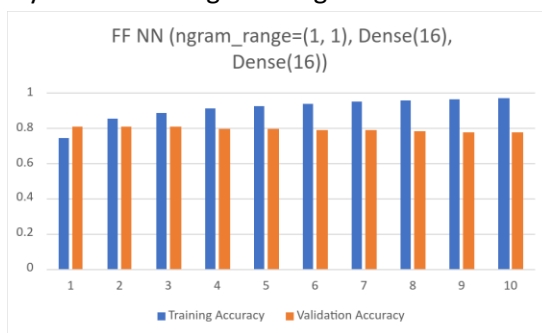
Word Count predictions

- Naïve Bayes – sklearn's `MultinomialNB` is used. The only hyperparameter is the n-gram range. I tried (1, 1), (1, 2), (1, 3), (2, 2), and (3, 3).



The training accuracy is inversely related to the testing accuracy. This clearly indicates that the model is overfitting as n increases. N-gram range of (1, 1) performs the best on the testing set which makes me think that the model learns the best on key words rather than sequences of words.

- Feedforward Neural Network – A Sequential model is used with two hidden layers. Every layer is a Dense layer. The first two layers use ReLu as the activation and softmax is used for the final layer. The optimizer selected is adam. The hyperparameters explored are the size of the hidden layers and the n-gram range.

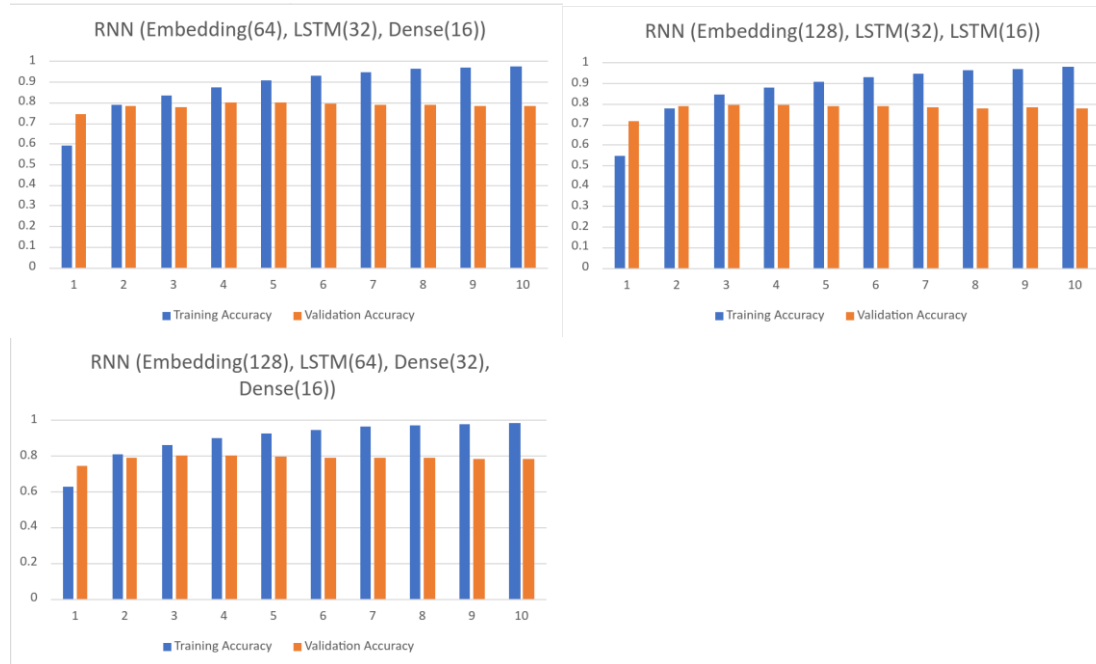


Regardless of the hyperparameters, each model quickly reaches close to 0.81 training accuracy in the first 1-3 epochs and then declines throughout the later epochs. This indicates that the function being learned is simpler than the models used and thus the models learn it quickly and then overfit. This can also be seen in the fact that each of them ends with a training accuracy close to or equal to 1.00.

Sequence predictions

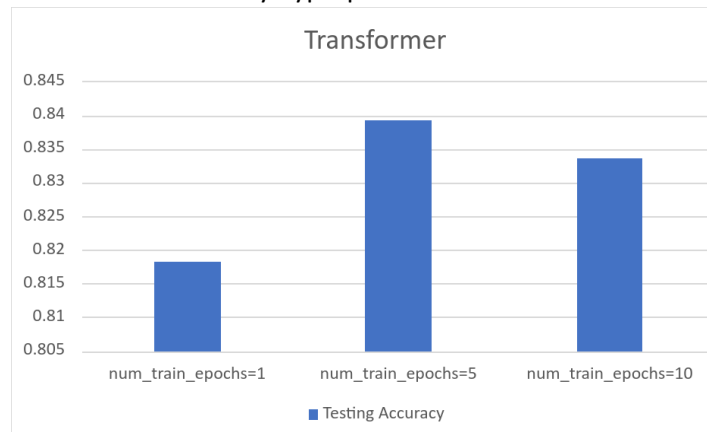
- Recurrent Neural Network – A Sequential model is used with two or three hidden layers and an embedding layer. The embedding layer finds word embeddings of length 64 or 128. The next

layer is always an LSTM. Each of the models has different layers following this. One uses a second LSTM. Another uses a single Dense layer. The last uses two Dense layers. Every Dense layer uses ReLu as the activation and softmax is used for the final layer. The optimizer selected is adam. The size of the hidden layers is adjusted as an additional hyperparameter.



Each model begins with a low testing accuracy of 0.71-0.74 that increases up to about 0.8 after five epochs and then proceeds to decline to around 0.78 after all ten epochs. This implies that they are overfitting after about five epochs. We can feel confident in this assumption given every model ends with a training accuracy around 0.98. I expected the model with two LSTMs to outperform the others, but it slightly underperforms relative to the others. It may be the case that this model is more complex than is necessary to fit the function.

- **Transformer** – The ClassificationModel from the simpletransformers package is used. The model used is the "bert-base-cased" model. The only hyperparameter is the number of training epochs.



The model attains 0.81 testing accuracy in the first epoch which is already better than every other model. After five epochs, it improves to 0.839 and then drops back down to 0.834 at ten

epochs, which implies that the model begins overfitting past five epochs.

Conclusion

During this project, I have explored several machine learning methods for predicting wine variety based on description and other features. As addressed in the K Neighbors section, the training data for the point predictions must have many points with identical locations but different varieties. Given this fact, 0.5 accuracy must be about the peak of what any model can achieve given the training data. In contrast, the description-based predictions consistently attained training accuracy greater than 0.7 with the transformer peaking at 0.839. I am not surprised that the transformer performed the best given the recent explosion of transformers being used for natural language processing tasks. The Feedforward Neural Network is not far behind with a peak accuracy of 0.81. I imagine that in the future, a significant amount of tuning could push it close to or past the transformer. The point-based models outperformed a random guess but were not particularly effective in their predictions. In contrast the description-based models were highly effective and could be used to semi-reliably predict a wine variety based on its review.